IMPERIAL

Department of Computing
70015: Mathematics for Machine Learning
Lecturer: Matthew Wicker

Lecture 03: Gradient-Based Optimization & Hessians

Additional Resources

NB: This lecture's additional materials will be a bit unhelpful to students until after Lecture 6. Divorcing GLMs from their probabilistic context is rarely done (though done in this course for pedagogical reasons). While I still provide them below, students are warned that these resources take a perspective we do not take until later in the course.

- Generalized linear models, Chapter 12 (page 445) Murphy (2012). Accessed via: https://probml.github.io/pml-book/book1.html
- Linear models for classification, Chapter 4 (page 179) Bishop (2006). Particularly, 4.1 - 4.1.3.

3.1 Advanced Linear Algebra & Vector Calculus

The introduction of a general non-linearity (as well as the inverse) requires us to move beyond the basic vector calculus rules we derived in the previous section. Here we will introduce what we call *advanced* vector calculus identities and rule. To do so, we will first introduce some new notation and objects that aid in such derivations.

3.1.1 Identity, Kronecker Delta, and Hadamard Product

The *canonical basis vector* $\mathbf{e}_i \in \mathbb{R}^n$ is the vector with a 1 in position i and 0 everywhere else. The identity matrix can be built from these as $\boldsymbol{I} = \sum_{i=1}^{n} \mathbf{e}_i \mathbf{e}_i^\top = [\mathbf{e}_1, \ldots, \mathbf{e}_n]$. Taking inner products of basis vectors gives the *Kronecker delta*:

$$\delta_{ij} = \mathbf{e}_i^\top \mathbf{e}_j = \begin{cases} 1 & i = j \\ 0 & i \neq j \end{cases}$$

so that $\boldsymbol{I}_{i,j} = \delta_{ij}$. The key property of the Kronecker delta is that it *sifts* sums: $\sum_j \delta_{ij}\, a_j = a_i$. It appears whenever we differentiate a variable with respect to itself, since $\partial \theta_i / \partial \theta_j = \delta_{ij}$, and

therefore $\partial\theta/\partial\theta = \boldsymbol{I}$. The definition extends to more indices; the three-axis Kronecker delta is

$$\delta_{ijk} = \begin{cases} 1 & i = j = k \\ 0 & \text{otherwise} \end{cases}$$

which can be thought of as the "identity" 3-tensor. Contracting one of its indices with a vector produces a diagonal matrix, $\sum_k \delta_{ijk}\, a_k = \text{diag}(\mathbf{a})_{i,j}$.

The *Hadamard product* of two matrices (or vectors) of the same shape is their elementwise product, $(\boldsymbol{A} \circ \boldsymbol{B})_{i,j} = \boldsymbol{A}_{i,j}\boldsymbol{B}_{i,j}$. For vectors, $(\mathbf{a} \circ \mathbf{b})_i = a_i b_i = \sum_{j,k} \delta_{ijk}\, a_j b_k$, or equivalently $\mathbf{a} \circ \mathbf{b} = \text{diag}(\mathbf{a})\, \mathbf{b}$.

Differentiating element-wise non-linearity Let $f : \mathbb{R} \to \mathbb{R}$ be some non-linear function. If we apply it element wise to a vector we will write $\boldsymbol{z} = f(\boldsymbol{x})$ for f applied to each entry, so $z_i = f(x_i)$. Now, the question becomes: what is the domain and co-domain of our function? It is relatively straight-forward to observe that when applied element-wise we have $f : \mathbb{R}^n \to \mathbb{R}^n$, thus we are looking for a matrix. Since z_i depends only on x_i, single-variable calculus tells us that:

$$\frac{\partial z_i}{\partial x_j} = f'(x_i)\, \delta_{ij} \quad \Longrightarrow \quad \frac{\partial \boldsymbol{z}}{\partial \boldsymbol{x}} = \text{diag}(f'(\boldsymbol{x})).$$

Differentiating the outer-product Let $\boldsymbol{F} = \boldsymbol{x}\boldsymbol{x}^\top$, so $\boldsymbol{F}_{i,j} = x_i x_j$. As always, we want to understand what the domain and co-domain of the operation is; in this case our input is only $\boldsymbol{x}$ so the input space is $\mathbb{R}^n$ and the output is in $\mathbb{R}^{n\times n}$, thus we should have a gradient of size $\mathbb{R}^{n\times n\times n}$. To do this, we can follow our previous steps and differentiate with respect to an arbitrary x_k, the product rule and $\partial x_i/\partial x_k = \delta_{ik}$ give:

$$\frac{\partial \boldsymbol{F}_{i,j}}{\partial x_k} = \delta_{ik}\, x_j + x_i\, \delta_{jk}.$$

This has three free indices, so it is a 3-tensor and cannot be reassembled into a matrix.

3.1.2 Computing the Gradient of Logistic Regression

We now have everything we need to compute the gradient of the binary logistic regression loss. For a design matrix $\boldsymbol{X} \in \mathbb{R}^{m\times(n+1)}$ and labels $\boldsymbol{y} \in \{0,1\}^m$, we can write the cross-entropy loss in vector form, with $\log$ and σ applied elementwise and $\mathbf{1}$ the vector of ones, as a composition of three functions:

$$\mathcal{L}(\theta) = \underbrace{-\frac{1}{m}\Big(\boldsymbol{y}^\top \log(\boldsymbol{s}) + (\mathbf{1}-\boldsymbol{y})^\top \log(\mathbf{1}-\boldsymbol{s})\Big)}_{\text{outer}}, \qquad \boldsymbol{s} = \underbrace{\sigma(\boldsymbol{z})}_{\text{middle}}, \qquad \boldsymbol{z} = \underbrace{\boldsymbol{X}\theta}_{\text{inner}}.$$

Following the composition $\theta \mapsto \boldsymbol{z} \mapsto \boldsymbol{s} \mapsto \mathcal{L}$, the chain rule gives

$$\nabla_\theta \mathcal{L} = \nabla_{\boldsymbol{s}}\mathcal{L} \cdot \nabla_{\boldsymbol{z}}\boldsymbol{s} \cdot \nabla_\theta \boldsymbol{z},$$

a $(1\times m)(m\times m)(m\times(n+1))$ product. We compute each factor in turn.

The outer factor. Writing $\boldsymbol{y}^\top \log(\boldsymbol{s}) = \sum_i y_i \log(s_i)$ and differentiating with respect to an arbitrary s_j, the Kronecker delta sifts out the $i = j$ term: $\sum_i y_i \, \delta_{ij}/s_i = y_j/s_j$. Treating the second term the same way,

$$\frac{\partial \mathcal{L}}{\partial s_j} = -\frac{1}{m}\left(\frac{y_j}{s_j} - \frac{1-y_j}{1-s_j}\right) = -\frac{1}{m}\,\frac{y_j - s_j}{s_j(1-s_j)}.$$

We collect these entries into the row vector $\nabla_{\boldsymbol{s}}\mathcal{L} = -\frac{1}{m}\boldsymbol{v}^\top$, where $v_j = (y_j - s_j)/\big(s_j(1-s_j)\big)$.

The middle factor. The logistic function has the convenient derivative

$$\sigma'(z) = \frac{e^{-z}}{(1+e^{-z})^2} = \frac{1}{1+e^{-z}} \cdot \frac{e^{-z}}{1+e^{-z}} = \sigma(z)\big(1-\sigma(z)\big),$$

so by the element-wise rule, $\partial \boldsymbol{s}/\partial \boldsymbol{z} = \operatorname{diag}\big(\boldsymbol{s} \circ (\boldsymbol{1} - \boldsymbol{s})\big)$.

The inner factor. $z_i = \sum_j \boldsymbol{X}_{i,j}\theta_j$ gives $\partial z_i/\partial \theta_k = \boldsymbol{X}_{i,k}$, and therefore $\partial \boldsymbol{z}/\partial \theta = \boldsymbol{X}$.

Combining. Multiplying the first two factors, the diagonal matrix becomes a Hadamard product, and the denominators of $\boldsymbol{v}$ cancel entry by entry:

$$\nabla_{\boldsymbol{s}}\mathcal{L}\,\frac{\partial \boldsymbol{s}}{\partial \boldsymbol{z}} = -\frac{1}{m}\big(\boldsymbol{v} \circ \boldsymbol{s} \circ (\boldsymbol{1} - \boldsymbol{s})\big)^\top = -\frac{1}{m}(\boldsymbol{y} - \boldsymbol{s})^\top.$$

Multiplying by the inner factor, we arrive at

$$\nabla_\theta \mathcal{L} = \frac{1}{m}\big(\sigma(\boldsymbol{X}\theta) - \boldsymbol{y}\big)^\top \boldsymbol{X}.$$

Writing out the product $(\cdot)^\top \boldsymbol{X}$ as a sum over rows recovers the summation form we stated earlier, $\nabla_\theta \mathcal{L} = -\frac{1}{m}\sum_i \big(y_i - \sigma(\boldsymbol{X}_{i,:}\theta)\big)\boldsymbol{X}_{i,:}$, now as a row vector. Compare this with the OLS gradient, $\frac{2}{m}(\boldsymbol{X}\theta - \boldsymbol{y})^\top \boldsymbol{X}$: in both cases the gradient is the prediction error weighted by the data. This is not a coincidence, and we will see why in a later lecture.

3.1.3 The Hessian of Logistic Regression

The *Hessian* of a scalar function $\mathcal{L} : \mathbb{R}^n \to \mathbb{R}$ is the matrix of its second partial derivatives, $(\nabla^2_\theta \mathcal{L})_{j,k} = \partial^2 \mathcal{L}/\partial\theta_j \partial\theta_k$. Equivalently, it is the Jacobian of the gradient: since our gradient is a row vector, we transpose it to a column and differentiate once more, $\nabla^2_\theta \mathcal{L} = \nabla_\theta(\nabla_\theta \mathcal{L})^\top$. For logistic regression, the previous section gives the transposed gradient as a composition of three functions:

$$(\nabla_\theta \mathcal{L})^\top = \underbrace{\frac{1}{m}\boldsymbol{X}^\top(\boldsymbol{s} - \boldsymbol{y})}_{\text{outer}}, \qquad \boldsymbol{s} = \underbrace{\sigma(\boldsymbol{z})}_{\text{middle}}, \qquad \boldsymbol{z} = \underbrace{\boldsymbol{X}\theta}_{\text{inner}}.$$

Since $\boldsymbol{X}$ and $\boldsymbol{y}$ are constant, the outer factor is simply $\frac{1}{m}\boldsymbol{X}^\top$, and the middle and inner factors are exactly those we computed for the gradient. The chain rule therefore gives

$$\nabla^2_\theta \mathcal{L} = \nabla_{\boldsymbol{s}}(\nabla_\theta \mathcal{L})^\top \cdot \nabla_{\boldsymbol{z}} \boldsymbol{s} \cdot \nabla_\theta \boldsymbol{z} = \frac{1}{m}\boldsymbol{X}^\top \boldsymbol{S}\boldsymbol{X}, \qquad \boldsymbol{S} = \operatorname{diag}\big(\boldsymbol{s} \circ (\boldsymbol{1} - \boldsymbol{s})\big),$$

or in index notation, $(\nabla^2_\theta \mathcal{L})_{j,k} = \frac{1}{m}\sum_i \boldsymbol{X}_{i,j}\, s_i(1-s_i)\, \boldsymbol{X}_{i,k}$, which is clearly symmetric. Notice that the labels $\boldsymbol{y}$ do not appear. More importantly, for any vector $\boldsymbol{u}$,

$$\boldsymbol{u}^\top \nabla^2_\theta \mathcal{L}\, \boldsymbol{u} = \frac{1}{m}\sum_i s_i(1-s_i)\,(\boldsymbol{X}\boldsymbol{u})_i^2 \geq 0,$$

since the logistic function maps into $(0,1)$, so each $s_i(1-s_i) > 0$. A Hessian with this property is called *positive semi-definite*, and it means the loss is *convex*: the loss is shaped like a bowl, a picture we will rely on when we study gradient descent, and any minimum we find is a global minimum.

3.2 Gradient Descent

Gradient descent is the core algorithm behind the *learning* portion of many machine learning algorithms. The concept behind this important algorithm is simple: if you want to find a/the minimum of a function, pick a random spot on the function and then iteratively take steps in the direction of steepest descent. Mathematically, the direction of steepest descent is the gradient of the function which we will denote with the ∇ operator. This operator will take a subscript to tell us what we are taking the gradient with respect to; for example, ∇_θ tells us to take the gradient of the proceeding function with respect to θ. Given this, we are now ready to state the most basic form of gradient descent:

$$\theta^{(t+1)} = \theta^{(t)} - \alpha \nabla_\theta \mathcal{L}(f^\theta, \boldsymbol{X}, \boldsymbol{y}), \tag{3.1}$$

where we assume without loss of generality that $\theta^{(0)} \sim \mathcal{N}(0,1)$ and that α is constant that we call the "learning rate". In what follows we will visualize how this iterative equation behaves for different selections of its user-defined parameters. Throughout this we will think of the loss function value, $\mathcal{L}(f^\theta, \boldsymbol{X}, \boldsymbol{y})$, as being a *height* or altitude that we would like to minimize. Specifically, visualizing the parameter θ as a puck, we will consider pushing the puck in a direction that minimizes its height.

As with our discussion of regularization, there are many opportunities for mathematical analysis of gradient descent, we will study these explicitly in future. For now, as with regression, we will build up our intuition by simply observing a few properties and visualizing the working of the algorithm.

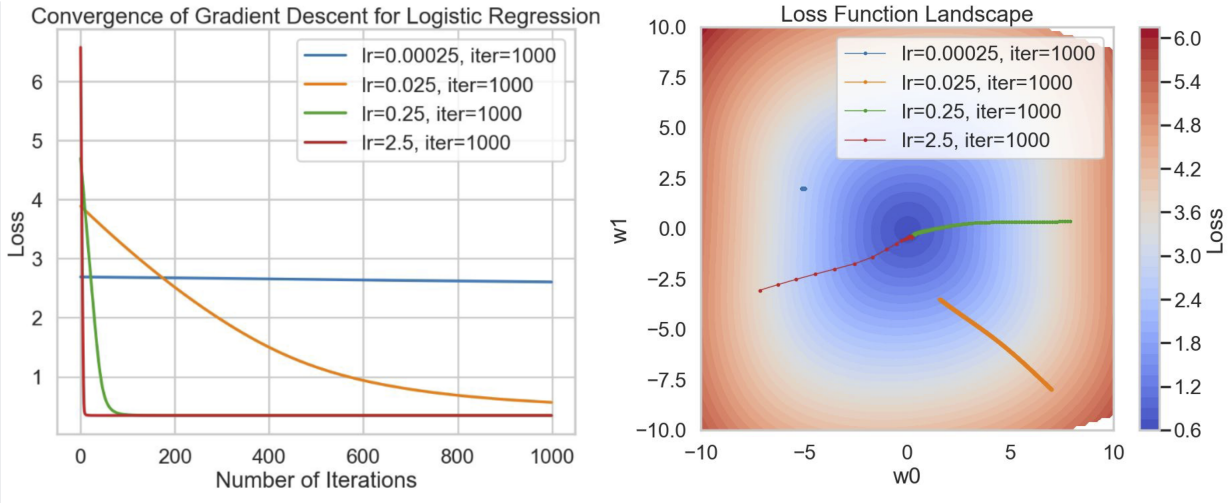


Figure 3.1: Effect of learning rate on the learning of our logistic regression model. **(a)** The convergence of the loss, we can interpret this as the *height* of the puck in the bowl. **(b)** A visualization of the path the parameter (puck) as it moves in the bowl.

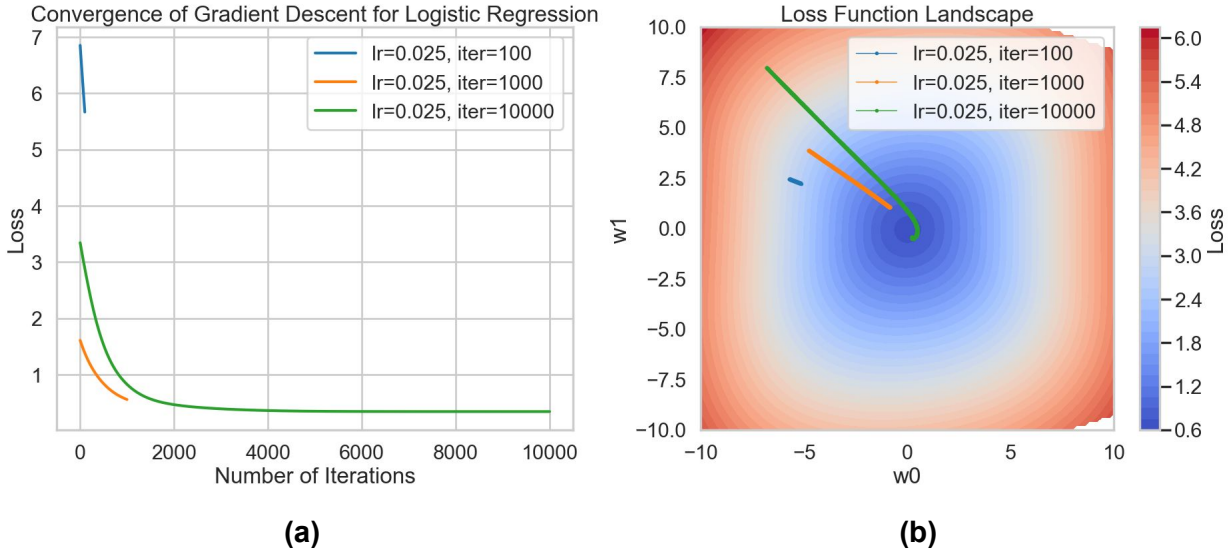


Figure 3.2: Effect of number of iterations on the learning of our logistic regression model. **(a)** The convergence of the loss, we can interpret this as the *height* of the puck in the bowl. **(b)** A visualization of the path the parameter (puck) as it moves in the bowl.

3.2.1 Complexity of Gradient Descent

With any learning algorithm selection, there are pros and cons. Now that we have seen two ways to minimize the mean squared error, we can imagine applying both to linear regression (remember we though we could compare OLS to GD in GLMs, it is a little bit indirect as we cannot use the OLS in GLMs). To start, lets consider the complexity of the OLS which takes the form: $\theta = (\boldsymbol{X}^\top \boldsymbol{X})^{-1}\boldsymbol{X}^\top \boldsymbol{y}$. The matrix product $\boldsymbol{X}^\top \boldsymbol{X}$ can be written in general as the product of matrices $\boldsymbol{AB}$ with $\boldsymbol{A} \in \mathbb{R}^{n\times l}$ and $\boldsymbol{B} \in \mathbb{R}^{l\times m}$. In this case, we have that the matrix-matrix product is given by the summation notation:

$$\boldsymbol{C}_{i,j} = \sum_{k=1}^{l} \boldsymbol{A}_{i,k}\boldsymbol{B}_{k,j}$$

Which has complexity $O(mln)$. Knowing that $\boldsymbol{C}$, in the case of $\boldsymbol{X}^\top \boldsymbol{X}$ will be of shape $n \times n$, and that each entry takes m multiplications and additions to compute, we have the total complexity of this operation as $O(n^2 m)$. Without too much discussion, we will allow students to use the fact that matrix inversion complexity is $O(n^3)$ where n is the dimension of the matrix's columns and rows (recall only square matrices are invertible). Thus we have that the computation of the matrix we want to invert is $O(n^2 m)$ and then the inversion is $O(n^3)$. Finally, we have the matrix-vector product at the end which is $\boldsymbol{c}_j = \sum_{i=1}^{n} \boldsymbol{Ab}$ which has complexity $O(mn)$ in the case of the final term in the OLS equation. So, overall, we have the complexity as $O(n^2 m + n^3)$.

In the case of gradient descent, we will often perform *stochastic* gradient descent which is where we sub-sample a *batch* of our data to compute the gradient. We will justify this mathematically in later lectures, but for now, let us assume $\boldsymbol{X'} \in \mathbb{R}^{b\times n}$. Recall that the gradient of linear regression is:

$$\left(\boldsymbol{X}^\top \boldsymbol{X}\theta - \boldsymbol{X}^\top \boldsymbol{y}\right)$$

As before, we have that the matrix multiplication is $O(b^2 n)$ (because we now have b rather than m samples) which dominates the matrix vector products which are done subsequently. At first glance, this seems hardly better than the complexity of the OLS! However, we can notice that the structure of our equation allows a much more efficient implementation i.e., by factoring you $\boldsymbol{X}^\top$ from our gradient computation we have:

$$\boldsymbol{X}^\top(\boldsymbol{X}\theta - \boldsymbol{y})$$

Looking inside the parentheses we have that $\boldsymbol{X}\theta$ is computed in $O(mn)$ time and returns a vector so when we multiply the result by $\boldsymbol{X}^\top$ we again have a complexity of $O(mn)$. When we consider that we have to compute the gradient k times this gives us a total gradient descent complexity of $O(kmn)$ with the batched setting being $O(kbn)$.

Comparing $O(kmn)$ with $O(n^2 m + n^3)$ one can immediately see that as long as $k < n$ we have that gradient descent will be more efficient than the OLS parameter from our previous lecture.

3.2.2 Visualizing Gradient Descent in Logistic Regression

In a later lecture we will perform a more formal mathematical analysis of gradient descent. Here, however, we will use a simple logistic regression task to visualize the working of gradient descent

for logistic regression. First, we have established a simple problem: two clouds of points representing each class.[1] We then run Equation (3.1) using different values of α and different terminal values of t. We highlight that, as with the regularization parameter λ, these values go into the *hyper-parameters* category that we will deal with later.

Effect of Learning Rate

Consider travelling along a bowl shaped landscape (which as we will see is a decent conceptual approximation). The learning rate can be conceptualized as how *fast* you are moving. In the case of Equation 3.1 the learning rate is also called the *step size*. In Figure 3.1 subplot b, we can see that there is a noticeable distance between red points indicating a large step size. Indeed the legend indicates that this is the largest learning rate we explored at $\alpha = 2.5$. Though each of our learning "trajectories" (i.e., paths through the loss landscape) start in a different place and therefore have a different starting loss, we can see in Figure 3.1 subplot a that the primary effect of the learning rate in this case is to speed up the rate at which we learn. Further discussion of this figure will take place in lectures.

Effect of Number of Iterations

Where we introduced the intuitive interpretation of the learning rate as the speed of our puck the number of iterations can be considered the amount of time we allow the puck to travel. We can see in Figure 3.2 that the longer we allow the puck to travel, the closer it gets to our optimal value.

3.2.3 Newton's Method

The Hessian describes the *curvature* of the loss. Near the current parameter θ, a second-order Taylor expansion gives a local quadratic model of the loss as a function of a step Δ:

$$\mathcal{L}(\theta + \Delta) \approx \mathcal{L}(\theta) + \nabla_\theta \mathcal{L}\, \Delta + \tfrac{1}{2} \Delta^\top \nabla_\theta^2 \mathcal{L}\, \Delta.$$

Gradient descent uses only the first-order term, and so must take small steps of a fixed size α. Instead, we can choose the step that minimizes the quadratic model exactly. Using the basic rules from the previous lecture (the linear rule and the quadratic form rule, with $\nabla_\theta^2 \mathcal{L}$ symmetric), the gradient of the model with respect to Δ is $\nabla_\theta \mathcal{L} + \Delta^\top \nabla_\theta^2 \mathcal{L}$. Setting this equal to zero and transposing gives $\nabla_\theta^2 \mathcal{L}\, \Delta = -(\nabla_\theta \mathcal{L})^\top$, and so we arrive at *Newton's method*:

$$\theta^{(t+1)} = \theta^{(t)} - \left(\nabla_\theta^2 \mathcal{L}\right)^{-1} (\nabla_\theta \mathcal{L})^\top.$$

Notice that there is no learning rate: the inverse Hessian sets the step size automatically, taking large steps along directions of low curvature and small steps along directions of high curvature. In the bowl picture, Newton's method does not care how elongated the bowl is.

[1] Using sklearn make classification function.

Newton's method for OLS. For the MSE loss, $(\nabla_\theta \mathcal{L})^\top = \frac{2}{m}\boldsymbol{X}^\top(\boldsymbol{X}\theta - \boldsymbol{y})$ and $\nabla^2_\theta \mathcal{L} = \frac{2}{m}\boldsymbol{X}^\top\boldsymbol{X}$. Substituting, a single Newton step from *any* starting point gives

$$\theta^{(1)} = \theta^{(0)} - (\boldsymbol{X}^\top\boldsymbol{X})^{-1}\boldsymbol{X}^\top(\boldsymbol{X}\theta^{(0)} - \boldsymbol{y}) = (\boldsymbol{X}^\top\boldsymbol{X})^{-1}\boldsymbol{X}^\top\boldsymbol{y},$$

which is exactly the OLS solution. This is no coincidence: the MSE loss *is* a quadratic, so the quadratic model is exact.

Newton's method for logistic regression. Substituting the gradient and Hessian from above (the factors of $\frac{1}{m}$ cancel), the update is

$$\theta^{(t+1)} = \theta^{(t)} - \big(\boldsymbol{X}^\top\boldsymbol{S}\boldsymbol{X}\big)^{-1}\boldsymbol{X}^\top(\boldsymbol{s} - \boldsymbol{y}), \qquad \boldsymbol{s} = \sigma(\boldsymbol{X}\theta^{(t)}), \quad \boldsymbol{S} = \operatorname{diag}\big(\boldsymbol{s} \circ (\mathbf{1} - \boldsymbol{s})\big).$$

With a little algebra this can be rewritten as $\theta^{(t+1)} = (\boldsymbol{X}^\top\boldsymbol{S}\boldsymbol{X})^{-1}\boldsymbol{X}^\top\boldsymbol{S}\,\tilde{\boldsymbol{y}}$, where $\tilde{\boldsymbol{y}} = \boldsymbol{X}\theta^{(t)} - \boldsymbol{S}^{-1}(\boldsymbol{s} - \boldsymbol{y})$. Compare this to the OLS solution: each Newton step solves a *weighted* least squares problem, with weights $\boldsymbol{S}$ and a modified target $\tilde{\boldsymbol{y}}$ that are recomputed at every iteration. For this reason, Newton's method for logistic regression is known as *iteratively reweighted least squares* (IRLS).

Trade-offs. Near the minimum, Newton's method converges extremely quickly, typically in a handful of iterations where gradient descent may need thousands. The price is the cost of each step: forming $\boldsymbol{X}^\top\boldsymbol{S}\boldsymbol{X}$ and solving the resulting linear system is $O(n^2m + n^3)$, the same as computing the OLS solution, compared to $O(mn)$ for a gradient step. Newton's method is therefore attractive when the number of parameters n is small, and impractical for models with millions of parameters. It also requires the Hessian to be invertible. For logistic regression this can fail when the classes are perfectly separable, since then the entries of $\boldsymbol{S}$ shrink toward zero as θ grows. A common fix is to add a small multiple of the identity to the Hessian, $\nabla^2_\theta \mathcal{L} + \epsilon\boldsymbol{I}$, which guarantees it is invertible; we will see in a later lecture that this corresponds to adding a *regularization* term to the loss.

Learning Objectives and Techniques

- Chain rule for differentiating complex functions
- Generalized linear models when we have output constraints
- Gradient descent
- Computing the complexity of matrix/vector/tensor operations

Bibliography

Bishop, C. M. (2006). Pattern recognition and machine learning. *Springer google scholar*, 2:5–43.

Deisenroth, M. P., Faisal, A. A., and Ong, C. S. (2020). *Mathematics for machine learning*. Cambridge University Press.

Murphy, K. P. (2012). *Machine learning: a probabilistic perspective*. MIT press.